Resources

linspace

Generate linearly spaced vector

Syntax

```
y = linspace(x1,x2)
y = linspace(x1,x2,n)
```

Description

`y = linspace(x1,x2)` returns a row vector of evenly spaced points between `x1` and `x2`. By default, `linspace` generates 100 points. example

`y = linspace(x1,x2,n)` generates `n` points. The spacing between the points is `(x2-x1)/(n-1)`. example

`linspace` is similar to the colon operator, ":", but gives direct control over the number of points and always includes the endpoints. "`lin`" in the name "`linspace`" refers to generating linearly spaced values as opposed to the sibling function `logspace`, which generates logarithmically spaced values.

Examples

collapse all

Vector of Evenly Spaced Numbers

Create a vector of 100 evenly spaced points in the interval `[-5,5]`.

Open in MATLAB Online

Copy Command

```
y = linspace(-5,5);
```

Get

Vector with Specified Number of Values

Create a vector of 7 evenly spaced points in the interval `[-5,5]`.

Open in MATLAB Online

Copy Command

```
y1 = linspace(-5,5,7)
```

Get

```
y1 = 1×7
```

```
    -5.0000   -3.3333   -1.6667         0    1.6667    3.3333    5.0000
```

Vector of Evenly Spaced Complex Numbers

Create a vector of complex numbers with 8 evenly spaced points between `1+2i` and `10+10i`.

Open in MATLAB Online

Copy Command

```
y = linspace(1+2i,10+10i,8)
```

Get

```
y = 1×8 complex

   1.0000 + 2.0000i   2.2857 + 3.1429i   3.5714 + 4.2857i   4.8571 + 5.4286i   6.1429 + 6.5714i
```

Input Arguments

collapse all

`x1,x2` — Point interval
pair of scalars

Point interval, specified as a pair of scalars. `x1` and `x2` define the interval over which `linspace` generates points. `x2` can be either larger or smaller than `x1`. If `x2` is smaller than `x1`, then the vector contains descending values.

Data Types: `single` | `double` | `datetime` | `duration`
Complex Number Support: Yes

`n` — Number of points
100 (default) | real numeric scalar | `NaN`

Number of points, specified as a real numeric scalar or `NaN`.

- If `n` is `1`, `linspace` returns `x2`.
- If `n` is zero or negative, `linspace` returns an empty 1-by-0 matrix.
- If `n` is not an integer, `linspace` rounds down and returns `floor(n)` points.
- If `n` is `NaN`, `linspace` returns `NaN`.

Extended Capabilities

collapse all

C/C++ Code Generation
Generate C and C++ code using MATLAB® Coder™.

Usage notes and limitations:

- Calls to `linspace` with number of points equal to `NaN` are not supported.

GPU Code Generation
Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Refer to the usage notes and limitations in the C/C++ Code Generation section. The same usage notes and limitations apply to GPU code generation.

Thread-Based Environment
Run code in the background using MATLAB® `backgroundPool` or accelerate code with Parallel Computing Toolbox™ `ThreadPool`.

The `linspace` function fully supports thread-based environments. For more information, see Run MATLAB Functions in Thread-Based Environment.

GPU Arrays
Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

The `linspace` function supports GPU array input with these usage notes and limitations:

- To run this function on a GPU and obtain a `gpuArray` output, use any of the following syntaxes:

```
y = gpuArray.linspace(x1,x2)
y = gpuArray.linspace(x1,x2,n)
```

- Alternatively, you can specify `x1` or `x2` as a `gpuArray`.

For more information, see Run MATLAB Functions on a GPU (Parallel Computing Toolbox).

Distributed Arrays
Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

The `linspace` function supports distributed arrays with these usage notes and limitations:

- Use `distributed.linspace` to call the distributed version of `linspace`.
- `x1` and `x2` must be `single` or `double` scalars.

For more information, see Run MATLAB Functions with Distributed Arrays (Parallel Computing Toolbox).

Version History

Introduced before R2006a

See Also

`logspace` | `colon`